

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 7

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan Algoritma Sorting. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman sort di Java yang meliputi Insertion Sort, Bubble Sort, dan Selection Sort

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan Algoritma Sorting di Java.

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum.....	3
1.2 Tujuan Praktikum.....	3
1.3 Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
2.1 InsertionSort_2511532010.....	5
2.2 BubbleSort_2511532010.....	7
2.3 SelectionSort_2511532010	9
2.4 InsertionSortGUI_2511532010.....	12
BAB III KESIMPULAN.....	19
3.1 Ringkasan Hasil Praktikum.....	19
DAFTAR PUSTAKA	20
TUGAS	21
1. Mahasiswa_2511532010.....	21
2. MahasiswaGUI_2511532010.....	22

BAB I PENDAHULUAN

1.1 Latar Belakang Praktikum

Pengurutan data (*sorting*) merupakan operasi fundamental dalam ilmu komputer yang esensial untuk mempermudah pencarian dan pengelolaan informasi, namun proses manipulasinya di dalam memori seringkali bersifat abstrak dan sulit dibayangkan. Pembelajaran algoritma dasar seperti *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* umumnya dilakukan melalui antarmuka konsol berbasis teks yang hanya menampilkan kondisi awal dan hasil akhir, tanpa memperlihatkan bagaimana proses pergeseran data terjadi. Oleh karena itu, praktikum ini dilaksanakan tidak hanya untuk mengimplementasikan logika murni dari ketiga algoritma tersebut menggunakan bahasa Java, tetapi juga untuk merancang aplikasi antarmuka grafis (GUI) yang interaktif. Melalui pendekatan visual ini, proses eksekusi algoritma dapat dikendalikan dan diamati secara bertahap (*step-by-step*), sehingga mampu menjembatani pemahaman konseptual algoritma dengan implementasi praktis pemrograman *event-driven* pada pembuatan aplikasi desktop.

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah agar praktikan mampu memahami alur kerja dan mengimplementasikan algoritma pengurutan *Insertion Sort*, *Bubble Sort*, dan *Selection Sort* ke dalam bentuk kode program Java yang fungsional. Selain itu, praktikum ini juga bertujuan untuk melatih keterampilan praktikan dalam merancang antarmuka grafis menggunakan komponen Java Swing, mengatur tata letak visual (*layout*), serta menerapkan pemrograman berbasis interaksi pengguna (*event-driven programming*). Secara khusus, praktikan dituntut untuk mampu memodifikasi logika perulangan algoritma yang biasanya berjalan otomatis menjadi sebuah sistem visual yang bisa dieksekusi langkah demi langkah oleh pengguna, lengkap dengan fitur validasi input, visualisasi array, dan pencatatan riwayat (*log*) proses pengurutan.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah peningkatan ketajaman logika pemrograman praktikan dalam menganalisis bagaimana sekumpulan data dibandingkan, digeser, dan ditukar posisinya secara komputasional di dalam memori. Praktikan juga mendapatkan pengalaman langsung dalam menyusun aplikasi desktop secara komprehensif, mulai dari penulisan kode *backend* (logika) hingga perancangan *frontend* (antarmuka pengguna) yang responsif dan tahan terhadap *error*. Lebih jauh lagi, program simulasi berbasis GUI yang dihasilkan sangat bermanfaat untuk melatih kemampuan pelacakan data (*debugging*) secara iteratif, sekaligus dapat dimanfaatkan sebagai media pembelajaran interaktif yang efektif bagi siapa saja yang ingin mempelajari cara kerja algoritma secara konkret dan transparan.

BAB II PEMBAHASAN

2.1 InsertionSort_2511532010

```
1 package pekan7_2511532010;
2
3 public class InsertionSort_2511532010 {
4
5     public static void insertionSort_2010(int[] arr_2010) {
6         int n_2010 = arr_2010.length;
7         for (int i_2010 = 1; i_2010 < arr_2010.length; i_2010++) {
8             int key_2010 = arr_2010[i_2010];
9             int j_2010 = i_2010 - 1;
10
11             while (j_2010 >= 0 && arr_2010[j_2010] > key_2010) {
12                 arr_2010[j_2010 + 1] = arr_2010[j_2010];
13                 j_2010--;
14             }
15             arr_2010[j_2010 + 1] = key_2010;
16         }
17     }
18
19     public static void main(String[] args_2010) {
20         int[] arr_2010 = {23, 78, 45, 8, 32, 56, 1};
21         int n_2010 = arr_2010.length;
22
23         System.out.printf("array yang belum terurut: \n");
24         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
25             System.out.print(arr_2010[i_2010] + " ");
26         }
27         System.out.println();
28
29         // Memanggil fungsi sorting
30         insertionSort_2010(arr_2010);
31
32         System.out.printf("array yang terurut: \n");
33         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
34             System.out.print(arr_2010[i_2010] + " ");
35         }
36         System.out.println();
37     }
38 }
```

1. Deklarasi Package dan Kelas

Program ini dibungkus dalam package pekan7_2511532010 dan menggunakan kelas bernama InsertionSort_2511532010. Terlihat bahwa penamaan variabel dan fungsi menggunakan akhiran _2010, yang kemungkinan merepresentasikan 4 digit terakhir Nomor Induk Mahasiswa (NIM).

2. Method insertionSort_2010 (Logika Pengurutan)

Ini adalah metode statis yang menerima parameter berupa array integer (arr_2010) dan bertugas untuk mengurutkan angka-angka tersebut dari yang terkecil ke yang terbesar (*ascending*).

3. Pemilihan Elemen Kunci (Outer Loop)

Perulangan for dimulai dari $i_2010 = 1$ (elemen kedua array) hingga elemen terakhir. Algoritma ini mengasumsikan elemen pertama (indeks 0) sudah berada di tempat yang benar. Elemen yang sedang dievaluasi disimpan ke dalam variabel `key_2010`.

4. Pengecekan ke Belakang (Inner Loop)

Variabel $j_2010 = i_2010 - 1$ digunakan untuk menunjuk elemen yang berada tepat di sebelah kiri `key_2010`. Perulangan while akan berjalan selama j belum melewati batas kiri array ($j \geq 0$) dan elemen di sebelah kiri lebih besar daripada `key_2010` (`arr_2010[j_2010] > key_2010`).

5. Pergeseran Posisi

Di dalam perulangan while, jika elemen kiri lebih besar, elemen tersebut akan "digeser" satu langkah ke kanan (`arr_2010[j_2010 + 1] = arr_2010[j_2010]`). Kemudian indeks j dikurangi satu (j_2010--) untuk mengecek elemen kirinya lagi. Proses ini terus berulang sampai ditemukan posisi yang tepat untuk `key_2010`.

6. Penyisipan Elemen

Setelah pergeseran selesai, nilai `key_2010` diletakkan di posisi kosong yang sudah disiapkan (`arr_2010[j_2010 + 1] = key_2010`).

7. Fungsi main (Inisialisasi Data)

Sebagai titik awal berjalannya program, sebuah array acak dibuat dengan elemen {23, 78, 45, 8, 32, 56, 1}.

8. Mencetak Array Awal

Program menggunakan perulangan for untuk mencetak kondisi array mentah ke layar dengan tulisan "array yang belum terurut:".

9. Eksekusi Pengurutan

Perintah `insertionSort_2010(arr_2010);` dipanggil. Karena array di Java bersifat *pass-by-reference* untuk isinya, array asal tersebut langsung dimodifikasi urutannya di dalam memori.

10. Mencetak Hasil Akhir

Program kembali menggunakan perulangan for untuk mencetak array yang sama. Karena array sudah melewati proses pengurutan di langkah 9, output di layar

sekarang akan menampilkan teks "array yang terurut:" diikuti dengan susunan angka yang benar (1, 8, 23, 32, 45, 56, 78).

2.2 BubbleSort_2511532010

```
1 package pekan7_2511532010;
2
3 public class BubbleSort_2511532010 {
4
5     public static void bubbleSort_2010(int[] arr_2010) {
6         int n_2010 = arr_2010.length;
7
8         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
9             for (int j_2010 = 0; j_2010 < n_2010 - i_2010 - 1; j_2010++) {
10
11                 if (arr_2010[j_2010] > arr_2010[j_2010 + 1]) {
12                     int temp_2010 = arr_2010[j_2010];
13                     arr_2010[j_2010] = arr_2010[j_2010 + 1];
14                     arr_2010[j_2010 + 1] = temp_2010;
15                 }
16             }
17         }
18     }
19
20     public static void main(String[] args_2010) {
21         int[] arr_2010 = {23, 78, 45, 8, 32, 56, 1};
22         int n_2010 = arr_2010.length;
23
24         System.out.print("array yang belum terurut:\n");
25         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
26             System.out.print(arr_2010[i_2010] + " ");
27         }
28         System.out.println("");
29
30         bubbleSort_2010(arr_2010);
31
32         System.out.print("array yang terurut menggunakan BubbleSort:\n");
33         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
34             System.out.print(arr_2010[i_2010] + " ");
35         }
36         System.out.println("");
37     }
38 }
```

1. Deklarasi Package dan Kelas

Program ini berada di dalam package pekan7_2511532010 dengan nama kelas BubbleSort_2511532010. Variabel-variabel di dalamnya juga konsisten menggunakan akhiran _2010 sebagai penanda.

2. Method bubbleSort_2010 (Logika Pengurutan)

Ini adalah metode statis yang bertugas mengurutkan array menggunakan algoritma Bubble Sort. Konsep dasar algoritma ini adalah membandingkan dua elemen

yang bersebelahan secara terus-menerus dan menukarnya jika urutannya salah, sehingga elemen terbesar akan "menggelembung" (bergeser) ke posisi paling kanan.

3. Perulangan Luar (Outer Loop)

Perulangan for yang pertama (`i_2010`) berfungsi untuk menentukan berapa kali proses penyisiran (pengecekan) array harus dilakukan. Perulangan ini akan berjalan sebanyak jumlah elemen array.

4. Perulangan Dalam (Inner Loop)

Perulangan for kedua (`j_2010`) berfungsi untuk melakukan perbandingan antar elemen yang bersebelahan dari kiri ke kanan. Batas perulangan ini menggunakan optimasi $n_2010 - i_2010 - 1$. Karena setiap kali satu putaran perulangan luar selesai maka elemen paling besar pasti sudah berada di ujung kanan, maka kita tidak perlu mengecek elemen bagian kanan yang sudah pasti berurut.

5. Kondisi Pengecekan dan Penukaran (Swapping)

Di dalam inner loop terdapat blok if yang mengecek apakah elemen saat ini (`arr[j]`) lebih besar dari elemen di sebelah kanannya (`arr[j + 1]`). Jika iya, posisi kedua angka tersebut ditukar.

6. Variabel Bantuan (`temp_2010`)

Untuk menukar posisi dua elemen pada langkah 5, program menggunakan variabel sementara bernama `temp_2010` agar nilai elemen pertama tidak hilang/tertimpa saat proses pertukaran data berlangsung.

7. Fungsi main (Inisialisasi Data)

Program utama dimulai dengan mendefinisikan array angka tidak berurut: {23, 78, 45, 8, 32, 56, 1}.

8. Mencetak Array Awal

Program menggunakan perulangan for untuk mencetak array mentah ke layar dengan menampilkan teks "array yang belum terurut:".

9. Eksekusi Pengurutan

Program memanggil fungsi `bubbleSort_2010(arr_2010)`. Di dalam memori, isi dari array tersebut langsung dimodifikasi dan ditukar posisinya hingga menjadi urut dari kecil ke besar.

10. Mencetak Hasil Akhir

Terakhir, program mencetak teks "array yang terurut menggunakan BubbleSort:" lalu menggunakan perulangan for untuk menampilkan isi array yang sekarang posisinya sudah benar (1, 8, 23, 32, 45, 56, 78).

2.3 SelectionSort_2511532010

```
1 package pekan7_2511532010;
2
3 public class SelectionSort_2511532010 {
4
5     public static void selectionSort_2010(int[] arr_2010) {
6         int n_2010 = arr_2010.length;
7
8         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
9             int minIndex_2010 = i_2010;
10
11             for (int j_2010 = i_2010 + 1; j_2010 < n_2010; j_2010++) {
12                 if (arr_2010[j_2010] < arr_2010[minIndex_2010]) {
13                     minIndex_2010 = j_2010;
14                 }
15             }
16
17             int temp_2010 = arr_2010[i_2010];
18             arr_2010[i_2010] = arr_2010[minIndex_2010];
19             arr_2010[minIndex_2010] = temp_2010;
20         }
21     }
22
23     public static void main(String[] args) {
24         int arr_2010 [] = {23, 78, 45, 8, 32, 56, 1};
25         int n_2010 = arr_2010.length;
26
27         System.out.printf("array yang belum terurut:\n");
28         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
29             System.out.print(arr_2010[i_2010] + " ");
30         }
31         System.out.println();
32
33         selectionSort_2010(arr_2010);
34
35         System.out.printf("array yang terurut:\n");
36         for (int i_2010 = 0; i_2010 < n_2010; i_2010++) {
37             System.out.print(arr_2010[i_2010] + " ");
38         }
39         System.out.println();
40     }
41 }
42
43 }
```

1. Deklarasi Package dan Kelas

Program ini dibungkus dalam package pekan7_2511532010 dan menggunakan nama kelas SelectionSort_2511532010. Variabel-variabel di dalamnya juga tetap konsisten menggunakan akhiran _2010.

2. Method selectionSort_2010 (Logika Pengurutan)

Metode statis ini menerapkan algoritma Selection Sort. Cara kerjanya adalah dengan mencari nilai terkecil dari kumpulan elemen yang belum terurut, lalu memindahkannya ke posisi paling depan secara bertahap.

3. Perulangan Luar (Outer Loop)

Perulangan for pertama (`i_2010`) menentukan batas area array yang sudah terurut dan yang belum. Perulangan ini berjalan dari indeks paling awal (0) hingga akhir array untuk menetapkan elemen demi elemen di posisi yang benar.

4. Penentuan Indeks Minimum Awal

Pada awal setiap putaran perulangan luar, program mengasumsikan bahwa elemen di posisi `i_2010` adalah nilai terkecil sementara. Indeks dari posisi ini disimpan ke dalam variabel `minIndex_2010`.

5. Perulangan Dalam (Pencarian Nilai Terkecil Sebenarnya)

Perulangan for kedua (`j_2010`) berjalan menyusuri sisa elemen di sebelah kanan `i_2010` (dimulai dari `i_2010 + 1`). Tujuannya adalah mencari apakah ada angka yang lebih kecil dari asumsi awal kita tadi.

6. Pembaruan Indeks Minimum

Di dalam inner loop terdapat pengecekan if. Jika program menemukan angka yang lebih kecil dari angka pada `minIndex_2010`, maka variabel `minIndex_2010` akan diperbarui dengan posisi (`j_2010`) angka baru yang lebih kecil tersebut.

7. Proses Penukaran (Swapping)

Setelah seluruh sisa elemen di sebelah kanan selesai dicek, program sudah pasti mengetahui di mana posisi angka terkecil berada (`minIndex_2010`). Program kemudian menukar elemen di posisi `i_2010` dengan elemen di `minIndex_2010` menggunakan bantuan variabel `temp_2010`.

8. Fungsi main (Inisialisasi Data)

Program utama dimulai dengan menyiapkan array angka yang sama: {23, 78, 45, 8, 32, 56, 1}.

9. Mencetak Array Awal dan Eksekusi

Program mencetak angka-angka mentah ke layar dengan teks "array yang belum terurut:", kemudian memanggil fungsi `selectionSort_2010(arr_2010)` untuk memproses array tersebut di dalam memori.

10. Mencetak Hasil Akhir

Terakhir, program mencetak teks "array yang terurut:" dan menampilkan isi array yang posisinya sudah tersusun dari nilai terkecil hingga terbesar (1, 8, 23, 32, 45, 56, 78).

2.4 InsertionSortGUI_2511532010

```
1 package pekan7_2511532010;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.Font;
7
8 import javax.swing.BorderFactory;
9 import javax.swing.JButton;
10 import javax.swing.JFrame;
11 import javax.swing.JLabel;
12 import javax.swing.JOptionPane;
13 import javax.swing.JPanel;
14 import javax.swing.JScrollPane;
15 import javax.swing.JTextArea;
16 import javax.swing.JTextField;
17 import javax.swing.SwingConstants;
18 import javax.swing.SwingUtilities;
19 import javax.swing.border.EmptyBorder;
20
21 public class InsertionSortGUI_2511532010 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_2010;
25     private JLabel[] labelArray_2010;
26     private JButton stepButton_2010, resetButton_2010, setButton_2010;
27     private JTextField inputField_2010;
28     private JPanel panelArray_2010;
29     private JTextArea stepArea_2010;
30
31     private int i_2010 = 1, j_2010;
32     private boolean sorting_2010 = false;
33     private int stepCount_2010 = 1;
34
35     /**
36      * Create the frame.
37      */
38     public InsertionSortGUI_2511532010() {
39         setTitle("Insertion Sort Langkah per Langkah");
40         setSize(1000, 450); // Diperbesar agar muat dengan JTextArea(60 kolom)
41         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42         setLocationRelativeTo(null);
43         setLayout(new BorderLayout());
44
45         // PERBAIKAN: Menggunakan BorderLayout agar komponen input rapi dalam satu baris
46         JPanel inputPanel = new JPanel(new BorderLayout(10, 0));
47         inputPanel.setBorder(new EmptyBorder(10, 10, 10, 10)); // Memberi jarak margin
48
49         inputField_2010 = new JTextField(); // lebar kolom otomatis mengikuti area CENTER
50         setButton_2010 = new JButton("Set Array");
51
52         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma) : "), BorderLayout.WEST);
53         inputPanel.add(inputField_2010, BorderLayout.CENTER);
54         inputPanel.add(setButton_2010, BorderLayout.EAST);
55
56         // Panel array visual
57         panelArray_2010 = new JPanel();
58         panelArray_2010.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.CENTER, 10, 50));
59
60         // Panel kontrol
61         JPanel controlPanel = new JPanel();
62         stepButton_2010 = new JButton("Langkah Selanjutnya");
63         resetButton_2010 = new JButton("Reset");
64
65         stepButton_2010.setEnabled(false);
66         controlPanel.add(stepButton_2010);
67         controlPanel.add(resetButton_2010);
68
69         // Area teks untuk log langkah-langkah
70         stepArea_2010 = new JTextArea(8, 50); // Sedikit dikurangi agar seimbang
71         stepArea_2010.setEditable(false);
72         stepArea_2010.setFont(new Font("Monospaced", Font.PLAIN, 14));
73
74         JScrollPane scrollPane_2010 = new JScrollPane(stepArea_2010);
75         scrollPane_2010.setBorder(BorderFactory.createTitledBorder("Log Proses Sorting"));
76
77         add(inputPanel, BorderLayout.NORTH);
78         add(panelArray_2010, BorderLayout.CENTER);
79         add(controlPanel, BorderLayout.SOUTH);
80         add(scrollPane_2010, BorderLayout.EAST);
81
82         // Event Set Array
83         setButton_2010.addActionListener(e -> setArrayFromInput_2010());
84
85         // Event Langkah Selanjutnya
86         stepButton_2010.addActionListener(e -> performStep_2010());
87
88         // Event Reset
89         resetButton_2010.addActionListener(e -> reset_2010());
90     }
91
92     private void setArrayFromInput_2010() {
93         String text_2010 = inputField_2010.getText().trim();
94
95         if (text_2010.isEmpty())
96             return;
97     }
98 }
```

```

97
98     String[] parts_2010 = text_2010.split(",");
99
100     array_2010 = new int[parts_2010.length];
101
102     try {
103         for (int k_2010 = 0; k_2010 < parts_2010.length; k_2010++) {
104             array_2010[k_2010] = Integer.parseInt(parts_2010[k_2010].trim());
105         }
106     } catch (NumberFormatException e_2010) {
107         JOptionPane.showMessageDialog(
108             this,
109             "Masukkan hanya angka yang dipisahkan dengan koma!",
110             "Error",
111             JOptionPane.ERROR_MESSAGE
112         );
113         return;
114     }
115
116     i_2010 = 1;
117     stepCount_2010 = 1;
118     sorting_2010 = true;
119
120     stepButton_2010.setEnabled(true);
121     stepArea_2010.setText("");
122     panelArray_2010.removeAll();
123
124     labelArray_2010 = new JLabel[array_2010.length];
125
126     for (int k_2010 = 0; k_2010 < array_2010.length; k_2010++) {
127         labelArray_2010[k_2010] = new JLabel(String.valueOf(array_2010[k_2010]));
128         labelArray_2010[k_2010].setFont(new Font("Arial", Font.BOLD, 24));
129         labelArray_2010[k_2010].setBorder(
130             BorderFactory.createLineBorder(Color.BLACK)
131         );
132         labelArray_2010[k_2010].setPreferredSize(
133             new Dimension(50, 50)
134         );
135         labelArray_2010[k_2010].setHorizontalAlignment(
136             SwingConstants.CENTER
137         );
138         panelArray_2010.add(labelArray_2010[k_2010]);
139     }
140
141     panelArray_2010.revalidate();
142     panelArray_2010.repaint();
143 }
144
145 private void performStep_2010() {
146     if (i_2010 < array_2010.length && sorting_2010) {
147         int key_2010 = array_2010[i_2010];
148         j_2010 = i_2010 - 1;
149
150         StringBuilder stepLog_2010 = new StringBuilder();
151         stepLog_2010.append("Langkah ")
152             .append(stepCount_2010)
153             .append(": Masukkan ")
154             .append(key_2010)
155             .append("\n");
156
157         while (j_2010 >= 0 && array_2010[j_2010] > key_2010) {
158             array_2010[j_2010 + 1] = array_2010[j_2010];
159             j_2010--;
160         }
161
162         array_2010[j_2010 + 1] = key_2010;
163
164         updateLabels_2010();
165
166         stepLog_2010.append("Hasil: ")
167             .append(arrayToString_2010(array_2010))
168             .append("\n\n");
169
170         stepArea_2010.append(stepLog_2010.toString());
171
172         i_2010++;
173         stepCount_2010++;
174
175         if (i_2010 == array_2010.length) {
176             sorting_2010 = false;
177             stepButton_2010.setEnabled(false);
178             JOptionPane.showMessageDialog(this, "Sorting selesai!");
179         }
180     }
181 }
182
183 private void updateLabels_2010() {
184     for (int k_2010 = 0; k_2010 < array_2010.length; k_2010++) {
185         labelArray_2010[k_2010].setText(String.valueOf(array_2010[k_2010]));
186     }
187 }
188
189 private void reset_2010() {
190     inputField_2010.setText("");
191     panelArray_2010.removeAll();
192     panelArray_2010.revalidate();

```

```
193     panelArray_2010.repaint();
194
195     stepArea_2010.setText("");
196     stepButton_2010.setEnabled(false);
197
198     sorting_2010 = false;
199     i_2010 = 1;
200     stepCount_2010 = 1;
201
202
203     private String arrayToString_2010(int[] arr_2010) {
204         StringBuilder sb_2010 = new StringBuilder();
205
206         for (int k_2010 = 0; k_2010 < arr_2010.length; k_2010++) {
207             sb_2010.append(arr_2010[k_2010]);
208
209             if (k_2010 < arr_2010.length - 1) {
210                 sb_2010.append(", ");
211             }
212         }
213
214         return sb_2010.toString();
215     }
216
217     public static void main(String[] args_2010) {
218         SwingUtilities.invokeLater(() -> {
219             InsertionSortGUI_2511532010 gui_2010 = new InsertionSortGUI_2511532010();
220             gui_2010.setVisible(true);
221         });
222     }
223 }
```

1. Deklarasi Kelas dan Pewarisan (Inheritance)

Kelas InsertionSortGUI_2511532010 dibuat mewarisi kelas JFrame bawaan Java. Ini adalah langkah utama yang mengubah program konsol teks biasa menjadi program aplikasi desktop yang memiliki jendela antarmuka visual.

2. Variabel Status (State Variables)

Program mendeklarasikan variabel seperti `i_2010`, `j_2010`, `sorting_2010` (penanda apakah proses sedang berjalan), dan `stepCount_2010`. Variabel-variabel ini sangat penting karena algoritma tidak berjalan sekaligus sampai selesai, melainkan harus "mengingat" sudah sampai di langkah mana saat tombol diklik.

3. Variabel Komponen GUI

Dideklarasikan juga berbagai elemen pembuat antarmuka seperti tombol (JButton), kolom teks input (JTextField), area log teks (JTextArea), dan panel-panel wadah (JPanel) tempat elemen-elemen tersebut akan disusun.

4. Pengaturan Jendela Utama

Di dalam konstruktor, program menyetel kerangka dasar jendela. Jendela diberi judul "Insertion Sort Langkah per Langkah", ukuran disetel ke 1000x450 piksel, dan LayoutManager diatur menggunakan BorderLayout yang membagi layar menjadi area Utara, Selatan, Timur, Barat, dan Tengah.

5. Panel Input (Bagian Atas)

Sebuah panel dibuat di area Utara untuk menampung teks panduan, kotak teks tempat pengguna mengetik angka (dipisahkan koma), dan tombol "Set Array" untuk mengunci inputan.

6. Panel Visualisasi (Bagian Tengah)

panelArray_2010 disiapkan di area Tengah layar dengan tata letak FlowLayout. Di sinilah nantinya angka-angka yang dimasukkan akan ditampilkan dalam bentuk jejeran kotak-kotak secara horizontal.

7. Panel Kontrol (Bagian Bawah)

Panel ini diletakkan di area Selatan dan berisi tombol "Langkah Selanjutnya" (untuk menjalankan *sorting* satu per satu) dan tombol "Reset". Pada saat aplikasi baru dibuka, tombol "Langkah Selanjutnya" dinonaktifkan agar tidak terjadi error sebelum data dimasukkan.

8. Panel Log (Bagian Kanan)

Sebuah area teks besar (JTextArea) ditempatkan di area Timur dan dibungkus dengan JScrollPane (agar bisa digulir ke bawah). Area ini berfungsi sebagai "buku catatan" untuk menampilkan teks riwayat pergeseran angka setiap kali langkah sorting dilakukan.

9. Penambahan Event Listener

Ketiga tombol ("Set Array", "Langkah Selanjutnya", dan "Reset") dihubungkan dengan fungsinya masing-masing menggunakan sintaks aksi (e -> namaFungsi()). Ini membuat aplikasi bisa merespons ketika tombol-tombol tersebut diklik oleh pengguna.

10. Metode setArrayFromInput_2010 (Membaca Data)

Saat pengguna menekan tombol "Set Array", fungsi ini akan mengambil teks dari kotak input, memecahnya berdasarkan karakter koma (,), dan mengubah string tersebut menjadi tipe data array integer (array_2010).

11. Penanganan Kesalahan (Error Handling)

Di dalam proses membaca data, terdapat blok try-catch. Fitur ini memastikan jika pengguna tidak sengaja memasukkan huruf atau simbol selain angka, aplikasi tidak

akan *crash*, melainkan memunculkan jendela *pop-up* peringatan (JOptionPane) yang ramah.

12. Membangun Kotak Angka Visual

Jika input angka terkonfirmasi valid, fungsi ini akan mencetak sekumpulan komponen JLabel (berbentuk kotak teks bergaris batas hitam) sebanyak jumlah elemen array. Kotak-kotak ini diisi dengan angka, diperbesar font-nya, dan dimunculkan ke layar bagian tengah.

13. Metode performStep_2010 (Algoritma Bertahap)

Ini adalah modifikasi khusus dari algoritma *Insertion Sort*. Karena dikendalikan oleh klik tombol, fungsi ini hanya memproses satu putaran pergeseran saja setiap kali dipanggil. Fungsi akan mengambil satu angka sebagai key, menggeser angka-angka di sebelah kirinya yang lebih besar, lalu meletakkan key tersebut.

14. Pencatatan Riwayat (Logging)

Saat pergeseran angka (poin 13) terjadi, program menggunakan StringBuilder untuk menyusun kalimat laporan, contohnya "Langkah 1: Memasukkan 23" dan mencetak urutan array pada saat itu. Teks ini langsung dikirim ke panel log di sebelah kanan layar.

15. Metode updateLabels_2010 (Sinkronisasi Visual)

Setelah algoritma mengubah susunan array di dalam memori komputer, fungsi ini bertugas memperbarui tampilan layar. Teks angka di dalam kotak-kotak visual akan ditukar posisinya agar persis sama dengan kondisi memori terbaru.

16. Pengecekan Akhir Sorting

Di akhir fungsi performStep_2010, variabel indeks (i) akan ditambah satu. Jika nilainya sudah sama dengan panjang batas array, itu berarti pengurutan telah selesai. Program kemudian mematikan tombol "Langkah Selanjutnya" dan memunculkan *pop-up* dengan pesan "Sorting selesai!".

17. Metode reset_2010 (Membersihkan Layar)

Fungsi ini bertugas membersihkan seluruh layar jika tombol "Reset" diklik. Mulai dari mengosongkan kotak input teks, menghapus kotak angka dari layar tengah, menghapus teks di layar log, hingga mereset variabel perhitungan kembali ke 0 atau 1

18. Metode main (Menjalankan Program)

Titik masuk jalannya aplikasi. Perintah ini menggunakan `SwingUtilities.invokeLater` yang merupakan standar wajib di Java untuk memastikan antarmuka grafis (GUI) dibangun pada antrean proses khusus (Event Dispatch Thread). Hal ini menjamin aplikasi berjalan mulus, responsif, dan tidak mengalami *freeze* (macet).

BAB III KESIMPULAN

3.1 Ringkasan Hasil Praktikum

terhadap algoritma pengurutan data (*sorting*) merupakan landasan penting dalam logika pemrograman. Praktikan telah berhasil mengimplementasikan dan menganalisis cara kerja tiga algoritma fundamental, yaitu *Insertion Sort*, *Bubble Sort*, dan *Selection Sort* menggunakan bahasa Java. Ketiga algoritma tersebut memiliki pendekatan komputasi yang berbeda—mulai dari menyisipkan elemen ke posisi yang tepat, membandingkan dan menukar elemen bersebelahan secara terus-menerus, hingga mencari nilai minimum untuk ditempatkan di posisi paling depan. Penerapan dasar pada program berbasis konsol ini membuktikan bahwa abstraksi manipulasi memori dapat dikonversi menjadi baris kode bersarang (*nested loop*) yang terstruktur, efisien, dan fungsional.

Selain pemahaman logika murni, praktikum ini juga menyimpulkan bahwa visualisasi algoritma melalui antarmuka grafis (GUI) sangat efektif dalam membedah kompleksitas proses komputasi menjadi sesuatu yang interaktif dan mudah dipahami. Melalui pemanfaatan pustaka Java Swing dan pemrograman *event-driven*, algoritma *Insertion Sort* yang biasanya tereksekusi dalam hitungan milidetik berhasil dimodifikasi menjadi simulasi visual yang bisa dikendalikan langkah demi langkah (*step-by-step*). Pembuatan aplikasi desktop ini tidak hanya memperkuat pemahaman konseptual terhadap algoritma, tetapi juga melatih keterampilan praktis yang komprehensif, mulai dari membangun antarmuka pengguna yang responsif, melakukan validasi input (penanganan *error*), hingga melacak pergeseran data secara *real-time* yang sangat berguna untuk proses *debugging*.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorials — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Mahasiswa_2511532010

```
1 package pekan7_2511532010;
2
3
4 public class Mahasiswa_2511532010 {
5     private String nama_2010;
6     private String nim_2010;
7     private String prodi_2010;
8
9
10    public Mahasiswa_2511532010(String nama_2010, String nim_2010, String prodi_2010) {
11        this.nama_2010 = nama_2010;
12        this.nim_2010 = nim_2010;
13        this.prodi_2010 = prodi_2010;
14    }
15
16
17    public String getNama_2010() {return nama_2010;}
18    public String getNim_2010() {return nim_2010;}
19    public String getProdi_2010() {return prodi_2010;}
20
21
22    public void setNama_2010(String nama_2010) {this.nama_2010 = nama_2010; }
23    public void setNim_2010(String nim_2010) {this.nim_2010 = nim_2010;}
24    public void setProdi_2010(String prodi_2010) {this.prodi_2010 = prodi_2010;}
25
26    public String toString_2010() {return nama_2010 + " - " + nim_2010 + " (" + prodi_2010 + ")";}
27 }
28
```

2. MahasiswaGUI_2511532010

```

1 package pekan7_2511532010;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import java.awt.*;
6 import java.awt.event.ActionEvent;
7 import java.awt.event.ActionListener;
8 import java.util.ArrayList;
9
10 public class MahasiswaGUI_2511532010 extends JFrame {
11     // Komponen GUI Input
12     private JTextField txtNama_2010;
13     private JTextField txtNim_2010;
14     private JTextField txtProdi_2010;
15
16     // Komponen GUI Aksi & Pilihan
17     private JButton btnTambah_2010;
18     private JButton btnHapus_2010;
19     private JButton btnSort_2010;
20     private JComboBox<String> cbAlgoritma_2010;
21
22     // Komponen GUI Output Display
23     private JTable tabelMahasiswa_2010;
24     private DefaultTableModel modelTabel_2010;
25     private JTextArea txtLogVisualisasi_2010;
26
27     // Struktur Data Penampung Object Mahasiswa
28     private ArrayList<Mahasiswa_2511532010> listMahasiswa_2010;
29
30     // Constructor GUI
31     public MahasiswaGUI_2511532010() {
32         listMahasiswa_2010 = new ArrayList<>();
33         inisialisasiKomponen_2010();
34     }
35
36     private void inisialisasiKomponen_2010() {
37         setTitle("Program Pengurutan Nama Mahasiswa_2010");
38         setSize(850, 600);
39         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
40         setLocationRelativeTo(null);
41         setLayout(new BorderLayout(10, 10));
42
43         // 1. PANEL INPUT DATA (Bagian Atas)
44         JPanel panelInput_2010 = new JPanel(new GridLayout(4, 2, 5, 5));
45         panelInput_2010.setBorder(BorderFactory.createTitledBorder("Form Input Mahasiswa"));
46
47         panelInput_2010.add(new JLabel(" Nama Mahasiswa:"));
48         txtNama_2010 = new JTextField();
49         panelInput_2010.add(txtNama_2010);
50
51         panelInput_2010.add(new JLabel(" NIM Mahasiswa:"));
52         txtNim_2010 = new JTextField();
53         panelInput_2010.add(txtNim_2010);
54
55         panelInput_2010.add(new JLabel(" Program Studi:"));
56         txtProdi_2010 = new JTextField();
57         panelInput_2010.add(txtProdi_2010);
58
59         btnTambah_2010 = new JButton("Tambah Data");
60         btnHapus_2010 = new JButton("Hapus Data Terpilih");
61         panelInput_2010.add(btnTambah_2010);
62         panelInput_2010.add(btnHapus_2010);
63
64         // 2. PANEL KONTROL SORTING & TABEL DATA (Bagian Tengah)
65         JPanel panelTengah_2010 = new JPanel(new BorderLayout(5, 5));
66
67         // Sub-panel Kontrol Sorting
68         JPanel panelKontrol_2010 = new JPanel(new FlowLayout(FlowLayout.LEFT));
69         panelKontrol_2010.add(new JLabel("Pilih Algoritma:"));
70         String[] pilihanAlgoritma_2010 = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
71         cbAlgoritma_2010 = new JComboBox<>(pilihanAlgoritma_2010);
72         panelKontrol_2010.add(cbAlgoritma_2010);
73
74         btnSort_2010 = new JButton("Mulai Sorting");
75         panelKontrol_2010.add(btnSort_2010);
76         panelTengah_2010.add(panelKontrol_2010, BorderLayout.NORTH);
77
78         // Tabel Data Mahasiswa
79         String[] kolomTabel_2010 = {"Nama", "NIM", "Prodi"};
80         modelTabel_2010 = new DefaultTableModel(kolomTabel_2010, 0);
81         tabelMahasiswa_2010 = new JTable(modelTabel_2010);
82         JScrollPane scrollTabel_2010 = new JScrollPane(tabelMahasiswa_2010);
83         scrollTabel_2010.setPreferredSize(new Dimension(400, 200));
84         panelTengah_2010.add(scrollTabel_2010, BorderLayout.CENTER);
85
86         // 3. PANEL VISUALISASI LOG LOGIKA (Bagian Kanan/Bawah)
87         txtLogVisualisasi_2010 = new JTextArea();
88         txtLogVisualisasi_2010.setEditable(false);
89         txtLogVisualisasi_2010.setFont(new Font("Monospaced", Font.PLAIN, 12));
90         JScrollPane scrollLog_2010 = new JScrollPane(txtLogVisualisasi_2010);
91         scrollLog_2010.setBorder(BorderFactory.createTitledBorder("Visualisasi Proses Sorting Langkah-demi-Langkah"));
92         scrollLog_2010.setPreferredSize(new Dimension(400, 200));
93
94         // Menggabungkan Semua Panel ke Frame Utama
95         add(panelInput_2010, BorderLayout.NORTH);
96         add(panelTengah_2010, BorderLayout.CENTER);
97         add(scrollLog_2010, BorderLayout.SOUTH);
98

```

```

99 // --- REGISTRASI EVENT LISTENER / ACTION_2010 ---
100
101 // Aksi Tombol Tambah Data
102 btnTambah_2010.addActionListener(new ActionListener() {
103     @Override
104     public void actionPerformed(ActionEvent e) {
105         tambahDataMahasiswa_2010();
106     }
107 });
108
109 // Aksi Tombol Hapus Data Terpilih
110 btnHapus_2010.addActionListener(new ActionListener() {
111     @Override
112     public void actionPerformed(ActionEvent e) {
113         hapusDataMahasiswa_2010();
114     }
115 });
116
117 // Aksi Tombol Mulai Sorting
118 btnSort_2010.addActionListener(new ActionListener() {
119     @Override
120     public void actionPerformed(ActionEvent e) {
121         jalankanProsesSorting_2010();
122     }
123 });
124
125 // Method untuk menambah data ke ArrayList dan memperbarui Tabel
126 private void tambahDataMahasiswa_2010() {
127     String nama_2010 = txtNama_2010.getText().trim();
128     String nim_2010 = txtNim_2010.getText().trim();
129     String prodi_2010 = txtProdi_2010.getText().trim();
130
131     if (nama_2010.isEmpty() || nim_2010.isEmpty() || prodi_2010.isEmpty()) {
132         JOptionPane.showMessageDialog(this, "Semua data input harus diisi!", "Peringatan", JOptionPane.WARNING_MESSAGE);
133         return;
134     }
135
136     // Membuat objek baru ADT Mahasiswa
137     Mahasiswa_2010 mhs_2010 = new Mahasiswa_2010(nama_2010, nim_2010, prodi_2010);
138     listMahasiswa_2010.add(mhs_2010);
139
140     // Update Tampilan Tabel
141     modelTabel_2010.addRow(new Object[]{mhs_2010.getNama_2010(), mhs_2010.getNim_2010(), mhs_2010.getProdi_2010()});
142
143     // Bersihkan Field Input kembali
144     txtNama_2010.setText("");
145     txtNim_2010.setText("");
146     txtProdi_2010.setText("");
147     txtNama_2010.requestFocus();
148 }
149
150 // Method untuk menghapus baris data mahasiswa terpilih
151 private void hapusDataMahasiswa_2010() {
152     int barisTerpilih_2010 = tabelMahasiswa_2010.getSelectedRow();
153     if (barisTerpilih_2010 >= 0) {
154         listMahasiswa_2010.remove(barisTerpilih_2010);
155         modelTabel_2010.removeRow(barisTerpilih_2010);
156     } else {
157         JOptionPane.showMessageDialog(this, "Silakan pilih baris data pada tabel yang ingin dihapus!", "Info", JOptionPane.INFORMATION_MESSAGE);
158     }
159 }
160
161 // Method pemicu sorting berdasarkan pilihan ComboBox
162 private void jalankanProsesSorting_2010() {
163     if (listMahasiswa_2010.size() < 2) {
164         JOptionPane.showMessageDialog(this, "Data minimal harus berjumlah 2 untuk diurutkan!", "Info", JOptionPane.INFORMATION_MESSAGE);
165         return;
166     }
167
168     txtLogVisualisasi_2010.setText(""); // Reset log visualisasi teks
169     String algoTerpilih_2010 = (String) cbAlgoritma_2010.getSelectedItem();
170
171     if (algoTerpilih_2010.equals("Insertion Sort")) {
172         insertionSort_2010();
173     } else if (algoTerpilih_2010.equals("Selection Sort")) {
174         selectionSort_2010();
175     } else if (algoTerpilih_2010.equals("Bubble Sort")) {
176         bubbleSort_2010();
177     }
178
179     refreshTampilanTabel_2010();
180 }
181
182 // Helper untuk mengambil representasi deretan nama mahasiswa saat ini (untuk visualisasi)
183 private String dapatkanSnapshotNama_2010() {
184     StringBuilder sb_2010 = new StringBuilder("");
185     for (int i = 0; i < listMahasiswa_2010.size(); i++) {
186         sb_2010.append(listMahasiswa_2010.get(i).getNama_2010());
187         if (i < listMahasiswa_2010.size() - 1) {
188             sb_2010.append(", ");
189         }
190     }
191     sb_2010.append("]");
192     return sb_2010.toString();
193 }
194
195 // Menyinkronkan susunan di ArrayList ke GUI JTable setelah disorting
196

```

```

197 private void refreshTampilanTabel_2010() {
198     modelTabel_2010.setRowCount(0); // Kosongkan tabel lama
199     for (Mahasiswa_2511532010 mhs_2010 : listMahasiswa_2010) {
200         modelTabel_2010.addRow(new Object[]{mhs_2010.getNama_2010(), mhs_2010.getNim_2010(), mhs_2010.getProd_2010()});
201     }
202 }
203
204 // ===== METHOD ALGORITMA SORTING_2010 =====
205
206 // 1. Algoritma Insertion Sort
207 private void insertionSort_2010() {
208     txtLogVisualisasi_2010.append("=== INSERTION SORT ===\n");
209     txtLogVisualisasi_2010.append("Data Awal : " + dapatkanSnapshotNama_2010() + "\n");
210
211     int n_2010 = listMahasiswa_2010.size();
212     for (int i_2010 = 1; i_2010 < n_2010; i_2010++) {
213         Mahasiswa_2511532010 kunciMhs_2010 = listMahasiswa_2010.get(i_2010);
214         String kunciNama_2010 = kunciMhs_2010.getNama_2010();
215         int j_2010 = i_2010 - 1;
216
217         // Menggunakan compareToIgnoreCase() untuk penentuan Ascending (A-Z) tanpa sensitivitas huruf kapital
218         while (j_2010 >= 0 && listMahasiswa_2010.get(j_2010).getNama_2010().compareToIgnoreCase(kunciNama_2010) > 0) {
219             listMahasiswa_2010.set(j_2010 + 1, listMahasiswa_2010.get(j_2010));
220             j_2010--;
221         }
222         listMahasiswa_2010.set(j_2010 + 1, kunciMhs_2010);
223
224         // Log Visualisasi per langkah luar loop
225         txtLogVisualisasi_2010.append("Langkah " + i_2010 + " : " + dapatkanSnapshotNama_2010() + "\n");
226     }
227 }
228
229 // 2. Algoritma Selection Sort
230 private void selectionSort_2010() {
231     txtLogVisualisasi_2010.append("=== SELECTION SORT ===\n");
232     txtLogVisualisasi_2010.append("Data Awal : " + dapatkanSnapshotNama_2010() + "\n");
233
234     int n_2010 = listMahasiswa_2010.size();
235     for (int i_2010 = 0; i_2010 < n_2010 - 1; i_2010++) {
236         int indeksMin_2010 = i_2010;
237         for (int j_2010 = i_2010 + 1; j_2010 < n_2010; j_2010++) {
238             if (listMahasiswa_2010.get(j_2010).getNama_2010().compareToIgnoreCase(listMahasiswa_2010.get(indeksMin_2010).getNama_2010()) < 0) {
239                 indeksMin_2010 = j_2010;
240             }
241         }
242         // Swap / Tukar posisi objek data mahasiswa
243         Mahasiswa_2511532010 tempMhs_2010 = listMahasiswa_2010.get(indeksMin_2010);
244         listMahasiswa_2010.set(indeksMin_2010, listMahasiswa_2010.get(i_2010));
245         listMahasiswa_2010.set(i_2010, tempMhs_2010);
246
247         // Log Visualisasi per iterasi (Pass)
248         txtLogVisualisasi_2010.append("Pass " + (i_2010 + 1) + " : " + dapatkanSnapshotNama_2010() + "\n");
249     }
250 }
251
252 // 3. Algoritma Bubble Sort
253 private void bubbleSort_2010() {
254     txtLogVisualisasi_2010.append("=== BUBBLE SORT ===\n");
255     txtLogVisualisasi_2010.append("Data Awal : " + dapatkanSnapshotNama_2010() + "\n");
256
257     int n_2010 = listMahasiswa_2010.size();
258     for (int i_2010 = 0; i_2010 < n_2010 - 1; i_2010++) {
259         for (int j_2010 = 0; j_2010 < n_2010 - i_2010 - 1; j_2010++) {
260             if (listMahasiswa_2010.get(j_2010).getNama_2010().compareToIgnoreCase(listMahasiswa_2010.get(j_2010 + 1).getNama_2010()) > 0) {
261                 // Swap / Tukar elemen
262                 Mahasiswa_2511532010 tempMhs_2010 = listMahasiswa_2010.get(j_2010);
263                 listMahasiswa_2010.set(j_2010, listMahasiswa_2010.get(j_2010 + 1));
264                 listMahasiswa_2010.set(j_2010 + 1, tempMhs_2010);
265             }
266         }
267         // Log Visualisasi per iterasi (Pass) luar loop
268         txtLogVisualisasi_2010.append("Pass " + (i_2010 + 1) + " : " + dapatkanSnapshotNama_2010() + "\n");
269     }
270 }
271
272 // Main Method untuk menjalankan GUI Program
273 public static void main(String[] args) {
274     SwingUtilities.invokeLater(new Runnable() {
275         @Override
276         public void run() {
277             new MahasiswaGUI_2511532010().setVisible(true);
278         }
279     });
280 }
281 }

```

Hasil :

Form Input Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

Tambah Data

Hapus Data Terpilih

Pilih Algoritma:

Insertion Sort

Mulai Sorting

Nama	NIM	Prodi
gibran	1010	Sistem Informasi
habib	2010	informatika
hany	3019	kedokteran gigi
Krisna	1019	Sistem Informasi
Radhiana	2017	Kebidanan

Visualisasi Proses Sorting Langkah-demi-Langkah

=== INSERTION SORT ===

Data Awal : [habib, hany, gibran, Krisna, Radhiana]

Langkah 1 : [habib, hany, gibran, Krisna, Radhiana]

Langkah 2 : [gibran, habib, hany, Krisna, Radhiana]

Langkah 3 : [gibran, habib, hany, Krisna, Radhiana]

Langkah 4 : [gibran, habib, hany, Krisna, Radhiana]

Form Input Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

Tambah Data

Hapus Data Terpilih

Pilih Algoritma:

Selection Sort

Mulai Sorting

Nama	NIM	Prodi
gibran	1010	Sistem Informasi
habib	2010	informatika
hany	3019	kedokteran gigi
Krisna	1019	Sistem Informasi
Radhiana	2017	Kebidanan

Visualisasi Proses Sorting Langkah-demi-Langkah

=== SELECTION SORT ===

Data Awal : [gibran, habib, hany, Krisna, Radhiana]

Pass 1 : [gibran, habib, hany, Krisna, Radhiana]

Pass 2 : [gibran, habib, hany, Krisna, Radhiana]

Pass 3 : [gibran, habib, hany, Krisna, Radhiana]

Pass 4 : [gibran, habib, hany, Krisna, Radhiana]

Form Input Mahasiswa

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

Tambah Data

Hapus Data Terpilih

Pilih Algoritma:

Bubble Sort

Mulai Sorting

Nama	NIM	Prodi
gibran	1010	Sistem Informasi
habib	2010	Informatika
hany	1019	Kedokteran gigi
Krisna	1019	Sistem Informasi
Radhiana	2017	Kebidanan

Visualisasi Proses Sorting Langkah-demi-Langkah

=== BUBBLE SORT ===

Data Awal : [gibran, habib, hany, Krisna, Radhiana]

Pass 1 : [gibran, habib, hany, Krisna, Radhiana]

Pass 2 : [gibran, habib, hany, Krisna, Radhiana]

Pass 3 : [gibran, habib, hany, Krisna, Radhiana]

Pass 4 : [gibran, habib, hany, Krisna, Radhiana]

Penjelasan

1. Kelas Model Data (Mahasiswa_2511532010)

Kelas ini saya gunakan sebagai struktur data untuk menampung objek individu mahasiswa dengan tiga atribut privat: nama_2010, nim_2010, dan prodi_2010.

2. Constructor dan Method Akses

Saya menambahkan *constructor* untuk mengisi data di awal, method *getter-setter* untuk membaca dan mengubah data, serta method *toString_2010()* untuk mencetak data dalam format teks tertentu.

3. Kelas Utama GUI (MahasiswaGUI_2511532010)

Kelas ini mewarisi (*extends*) *JFrame* untuk membuat jendela aplikasi, mendeklarasikan seluruh elemen visual, dan menyatukannya ke dalam antarmuka.

4. Struktur Data Internal (ArrayList)

Saya menggunakan `ArrayList<Mahasiswa_2511532010>` sebagai penampung dinamis di memori untuk menyimpan seluruh objek data mahasiswa yang diinput.

5. Pengaturan Tata Letak (*inisialisasiKomponen_2010*)

Saya menyusun komponen visual menggunakan BorderLayout sebagai kerangka utama, GridLayout untuk merapikan form input, dan FlowLayout untuk menyelaraskan ComboBox serta tombol sortir.

6. Logika Tambah Data (tambahDataMahasiswa_2010)

Method ini bertugas membaca teks dari form, memvalidasi kolom kosong dengan `.isEmpty()`, membuat objek mahasiswa baru, memasukkannya ke `ArrayList`, lalu menambahkannya ke baris tabel visual `JTable`.

7. Logika Hapus Data (hapusDataMahasiswa_2010)

Fitur ini mendeteksi baris yang dipilih melalui `tabelMahasiswa_2010.getSelectedRow()`, kemudian menghapus data tersebut dari `ArrayList` internal dan menghapus barisnya dari tabel visual.

8. Pengendali Sorting (jalankanProsesSorting_2010)

Method ini melakukan validasi agar data minimal berjumlah 2, membaca algoritma yang dipilih pada ComboBox, mengeksekusi fungsi sorting yang sesuai, lalu memperbarui tabel lewat `refreshTampilanTabel_2010()`.

9. Algoritma Insertion Sort

Saya mengimplementasikannya dengan cara mengambil satu elemen kunci, membandingkannya secara mundur ke kiri, lalu menggeser elemen yang lebih besar ke kanan untuk menyisipkan elemen kunci pada posisi yang pas.

10. Algoritma Selection Sort

Algoritma ini memindai sisa array ke arah kanan untuk mencari nama mahasiswa yang secara alfabetis paling kecil, lalu melakukan pertukaran posisi (*swap*) ke depan pada setiap akhir putaran.

11. Algoritma Bubble Sort

Saya menggunakan perulangan bertingkat untuk membandingkan dua elemen yang berdampingan dan menukar posisinya secara langsung jika urutan abjadnya salah, hingga data terbesar bergeser ke posisi akhir.

12. Mekanisme Perbandingan String

Saya menerapkan fungsi `compareToIgnoreCase()` pada ketiga algoritma di atas agar pengurutan bersifat *Ascending* (A-Z) tanpa terpengaruh oleh perbedaan huruf kapital atau huruf kecil.

13. Log Visualisasi Langkah demi Langkah

Di dalam perulangan algoritma, saya memanggil fungsi `dapatkanSnapshotNama_2010()` untuk mengambil urutan nama saat itu dalam bentuk teks array dan langsung mencetaknya ke `JTextArea`.

14. Metode Utama (main method)

Fungsi ini bertugas menjalankan aplikasi pertama kali dan dibungkus dengan `SwingUtilities.invokeLater()` agar pembuatan GUI berjalan aman di dalam *Event Dispatch Thread*.</Mahasiswa_2511532010>